Chalmers | Göteborgs Universitet

Concurrent Programming TDA384/DIT392

17 March 2025

Exam supervisor: N. Piterman (piterman@chalmers.se, 031 772 6053)

(Exam set by N. Piterman and G. Schneider, based on the course given in Jan-Mar 2025)

Material permitted during the exam (hjälpmedel):
Two textbooks; four sheets of A4 paper with notes (single or double-sided); English dictionary (no smart phones allowed).

Grading: You can score a maximum of 70 points. Exam grades are: between 28–41 (3), between 42–55 (4), 56 or more (5).

Passing the course requires passing the exam and passing the labs. The overall grade for the course is determined as follows: between 40–59 (3), between 60-79 (4), 80 or more (5).

The exam **results** will be available in Ladok within 15 *working* days after the exam's date.

Instructions and rules:

- Please write your answers clearly and legibly: unnecessarily complicated solutions will lose points, and answers that cannot be read will receive no points!
- Justify your answers, and clearly state any assumptions that your solutions may depend on for correctness.
- Answer each question on a new page. Glance through the whole paper first; four questions, numbered Q1 through Q4. Do not spend more time on any question or part than justified by the points it carries.
- Be precise. In your answers, try to use the programming notation and syntax used in the questions. You can also use pseudo-code, *provided* the meaning is precise and clear. If need be, explain your notation.

This page is left empty intentionally.

Q1 (36p). In what follows you have 12 subquestions (Parts a to l) concerning different topics seen in the course. Each part a multiple-choice question. The grading for each question is as follows:

- For a right answer you will get 3 points;
- If you do not answer the question, you will get 0 points;
- If you answer wrongly, you will get -1 (a negative point).

The total amount of points will be done summing all the points for each part. So, if you answer correctly Part a, incorrectly Part b, and do not answer any of the rest, you will get 2 points (3 points for Part a minus 1 point for Part b, and 0 for the rest). Note that no negative points for the whole question Q1 will be given (the minimum number of points you may get for the whole question is 0). That is, if you do answer wrongly in each part, you will get 0 rather than -12.

NOTE: For each sub-question, exactly one option is correct.

(Part a). (3p) As part of your solution to Amazed, this is the way a worker adds a node to the set and then continues handling it:

```
10      public void addElement(int node)  {
11          if (!s.contains(node)) {
12              s.add(node);
13              // Now that I successfully added node to the set
14              // This is now *my* task to continue handling it
15              ....
16          }
17          // It was already there, I don't need to handle this node
18      }
```

a) This is a good example of using a concurrent set.

b) This creates a data race.

c) Multiple threads could handle the same node.

d) This could lead to a node not being handled at all.

Answer: Option c). If multiple threads check if a node is contained in the set they could all proceed to add the same node (not checking the success of the addition) and end up doing the same work for the same node.

(Part b). (3p)

Consider the following Erlang code:

```
4  main() ->
5      Self = self(),
6      First = spawn(fun () -> Self ! { self(), 2} end),
7      Second = spawn(fun () -> Self ! { self(), First } end),
8      receive
9          { Id, _ } ->
10             receive
11                 { Second, Id } -> ok;
12                 { Second, _ } -> notok
13             end
14     end.
```

a) It will always block.

b) It will either evaluate to `ok` or `notok`.

c) It will always evaluate to `ok`.

d) It will sometimes block and sometimes evaluate to `ok`.

Answer: Option d). The result depends on the order of arrival of the messages sent by `First` *and* `Second`*. If the message of* `First` *arrives first, then the* `pid` *of* `First` *is stored in* `Id`*. Then, the message from* `Second` *matches the pattern* `{ Second, Id }`*. If the message of* `Second` *arrives first, then the* `pid` *of* `Second` *is stored in* `Id`*. Then, the message from* `First` *(*`{ First, 2}`*) matches neither* `{ Second, Id }` *nor* `{ Second, _ }`*.*

(Part c). (3p)

An `AtomicReference` allows to perform `compare_and_set` on references. A `compare_and_set(expected,newval)` does two things atomically: if the value of the reference is `expected`, it sets the reference to `newval`. It returns `true` if it succeeds and `false` otherwise. The following code snippet shows one possible use of it:

```
11      AtomicReference<Integer> r =
12          new AtomicReference<Integer>(Integer.valueOf(i:0));
13
14      public void useAtomicReference() {
15          Integer j;
16          do {
17              j = r.get();
18              modify_integer_value(j);
19          } while (!r.compareAndSet(r.get(),j));
20      }
21
22      public void modify_integer_value(Integer j) {
23          // do some complex computation that changes j
24      }
```

a) The do-while loop always exits after one iteration.

b) The do-while loop exits only if `j` happens to be unmodified.

c) The do-while loop never exits.

d) The do-while loop depends on other threads that have access to `r`.

Answer: Option d). As `r` is a reference, its address does not change when changes are made to its content. So "our" thread is making changes only to the contents and not to the reference itself. However, other threads, may be changing the reference itself, in which case, the do-while loop might repeat depending on whether others are changing the contents or the reference.

(Part d). (3p)

Figure 1 shows a *wrong* Java implementation of a strong semaphore using Java's explicit scheduling mechanism (for suspending and resuming threads). Figure 2 shows the method calling the semaphore.

NOTE: `blocked` is a queue.

a) The program is wrong since it may deadlock.

b) The program is wrong since there are data races.

c) The program is wrong since `sem.up()` is never called.

d) The program is wrong because of all the above reasons.

Answer: Option a: The problem is that it might deadlock. The reason is the line checking whether `blocked` is empty before incrementing the counter in the `up()` method. As shown in Lecture 3 the fix is to remove the `if-then-else` in lines 3-4 (just increment and notify all)

```java
class SemaphoreStrong implements Semaphore {
    public synchronized void up()
    {   if (blocked.isEmpty()) count = count + 1;
        else notifyAll();    } // wake up all waiting threads

    public synchronized void down() throws InterruptedException
    {   Thread me = Thread.currentThread();
        blocked.add(me);  // enqueue me
        while (count == 0 || blocked.element() != me)
            wait();         // I'm enqueued when suspending
        // now count > 0 and it's my turn: dequeue me and decrement
        blocked.remove();  count = count - 1;   }

    private final Queue<Thread> blocked = new LinkedList<>();
```

Figure 1: A Java implementation of strong semaphores

```java
class StrongSemUser implements Runnable {
    private SemaphoreStrong sem = new SemaphoreStrong(1);

    public void run()
    {   while (true) {
            // Non critical
            sem.down();
            // Critical
            sem.up();
        }
    }
```

Figure 2: Run method calling the semaphore.

(Part e). (3p)

Let us consider the program of Part d above. You can only detect the error with:

a) Exactly one thread.

b) At least two threads.

c) Exactly three threads.

d) Exactly four threads.

e) None of the above.

Answer:

Option b: One thread is not enough. You need at least 2 threads (the red and the blue threads in the slides).

(Part f). (3p)

Let us consider the program of Part d above.

a) The error is in line 10 of the program in Figure 1: the threads should not wait.

b) The error is in line 9 of the program in Figure 1: one should replace `count == 0` by `count <> 0`.

c) The error is in the line checking whether `blocked` is empty before incrementing the counter in the `up()` method (lines 3 and 4).

d) The error is in the `run` method (Figure 2): you need to swap lines 21 and 23.

Answer:

Option c: The fix is to remove the `if-then-else` *in lines 3-4 (just increment and notify all); without that the program deadlocks.*

(Part g). (3p)

Let us consider the following code:

```
public class BarrierMonitor {
    private int expect;
    private int current;

    public BarrierMonitor(int n) {
        this.expect = n;
        this.current = 0;
    }

    public synchronized void await() throws InterruptedException  {
        current++;
        if (current<expect) {
            wait();
        }
        else {
            current = 0;
            notifyAll();
        }
    }
```

Does the class implement a barrier?

a) No.

b) Yes. Assuming that there are no spurious wakeups and interruptions.

c) Only under signalling policy signal and continue.

d) Only under signalling policy signal and wait.

e) Yes, but only if no more than n threads arrives at the same time.

Answer: Option b). The synchronization of the await method ensures that `current` *correctly captures the number of arrivals. Once it reaches expected, all are notified and continue to the next round.*

(Part h). (3p)

To find the total number of combinations of size R from a set of size N, where R is less than or equal to N, we use the following *combinatorial* formula:

$$\frac{N!}{R!\ (N-R)!}$$

Fig. 3 shows an Erlang parallel algorithm to compute the above formula. What is the maximal number of processes running in parallel when executing `parallelcomb` with parameters $N = 4$ and $R = 3$?

a) 1 (there is no parallelism)

b) 3

c) $ln(4)$ (ln is the natural logarithm)

d) 7 $(N + R)$

e) 4

f) 13 $((N * R) + (N - R))$

```
1   -module(parallel_combinations).
2   -export([parallelComb/2, start/0]).
3
4   parallelComb(N, R) when N >= R, R >= 0 ->
5       Parent = self(),
6       spawn(fun() -> Parent ! {n_fac, factorial(N)} end),
7       spawn(fun() -> Parent ! {r_fac, factorial(R)} end),
8       spawn(fun() -> Parent ! {nr_fac, factorial(N-R)} end),
9
10      N_Fac = receive {n_fac, Value} -> Value end,
11      R_Fac = receive {r_fac, Value} -> Value end,
12      NR_Fac = receive {nr_fac, Value} -> Value end,
13
14      N_Fac div (R_Fac * NR_Fac).
15
16  factorial(0) -> 1;
17  factorial(N) when N > 0 -> N * factorial(N - 1).
```

Figure 3: Erlang program `parallelComb`.

Answer: Options b) and e). Reason: the program launches three (non-recursive) processes, one for each call to the factorial function, which is a sequential function. So, there will be a maximum of four processes in parallel: three parallel processes + the parent process. The question was not clear enough whether we mean "running" as in not-blocked or as in process existing. We hence accepted both b) and e) as possible answers.

(Part i). (3p)

We have seen that according to Amdahl's law, if p is the fraction of a program that can be parallelised, then the maximum speedup that can be achieved by n processes is

$$\frac{1}{(1-p)+\frac{p}{n}}$$

We want to apply Amdahl's law to the Erlang program shown in Fig. 3 to calculate the speedup produced by this parallel version with respect to a purely sequential version of the program. Which one of the following assertions is true?

a) There is a speedup of a factor of 3, given that there are three spawned processes.

b) The sequential and the parallel version are essentially the same, so there is no speedup at all.

c) Amdahl's law is used to make a prediction about the maximum performance improvement that can be expected from parallelising a program, and not to compare two versions of the same program. So, the question doesn't make sense.

d) The parallel version can run at least 3 times faster than the purely sequential one.

e) None of the above.

Answer: Option c). Reason: Amdahl's law is not used to compare programs but to estimate the speedup if we know which percentage of the program can be parallelised.

(Part j). (3p)

Let us consider the monitor class in Figure 4:

Does the class implements a strong semaphore?

a) Only under signalling policy signal and continue.

b) Only under singnalling policy singal and wait.

```
monitor class StrongSemaphore implements Semaphore {
 int count;
 Condition isPositive = new Condition(); // is count > 0?

 public void down() {
   while (count > 0)
     count = count - 1;
   isPositive.wait();
 }


 public void up() {
   if (isPositive.isEmpty())
     count = count + 1;
   else isPositive.signal();
 }
}
```

Figure 4: Q1-j: Program `StrongSemaphore`

c) Yes. Assuming that there are no spurious wakeups and interruptions.

d) No.

Answer: Option d). Down reduces count to 0 and then waits. This is regardless of the initial value of count.

(Part k). (3p) Which one of the following assertions is true?

a) Data races are not possible in Erlang.

b) Race conditions are not possible in Erlang.

c) In Erlang neither data races nor race conditions are possible.

d) In Erlang both data races and race conditions are possible.

e) None of the above is correct.

Answer: Option a). There are no shared variables in Erlang. In particular, one cannot have two accesses to the same memory location.

(Part l). (3p)

We show in Figure 5 the pseudocode of part of a program `counting` (threads t and u). Let us assume that a main program launches the two threads and whenever both terminate, it prints the result of `counter`.

You should assume that interleaving only happens in between instructions, and not "inside" an instruction (e.g., between lines 2 and 3, but

```
                int counter = 0;     int i = 0;
          thread t                          thread u
 int cnt;                          int cnt;

1 while (i<50) {                   while (i<50) {              7
2   i = i+1;                         i = i+1;                  8
3   cnt = counter;                   cnt = counter;            9
4   counter = cnt + 1;               counter = cnt + 1;        10
5 }                                }                           11
6 // end                           // end                      12
```

Figure 5: Pseudocode of program `counting`.

not in line 2 — between reading the value of i, incrementing i and assigning the new updated value to i)

Which one of the following assertion is correct concerning the minimum and the maximum values the program can print (the value of `counter` after termination)?

a) Minimum = 1 and Maximum = 100.

b) Minimum = 1 and Maximum = 51.

c) Minimum = 1 and Maximum = 50.

d) Minimum = 2 and Maximum = 101.

e) None of the above is correct.

Answer: Option b:

Minimum: 1 (Thread t enters the loop once and stops in line 4, so i=1 and cnt=0; thread u executes the whole loop 50 times and exits with i=50 and counter=49 (since i is incremented once by t); thread t executes line 4 making counter=1 (since cnt=0) and then exits the loop).

Maximum: 51 (Thread t enters the loop and stops before executing line 2. Thread u executes the whole loop exiting with i=50 and counter=50. Thread t now executes lines 2-4, making counter=51.)

Q2 (12p). The *Smoothie Makers Problem* coordinates three smoothie makers. Each smoothie maker has an infinite supply of one of three smoothie ingredients: fruit, yogurt, and juice. A fourth person, the agent, supplies two of the three ingredients occasionally (each ingredient is supplied repeatedly). A smoothie maker that can get the two missing ingredients then makes a smoothie and enjoys it. For example, the smoothie maker that has infinite supply of fruit, needs to collect a portion of yogurt and a portion of juice (delivered earlier by the agent) and then make a smoothie out of the three. The challenge is to synchronize the smoothie makers and the agent. We give two suggested solutions: monitor based and semaphore based. For each solution you should identify whether it solves the problem correctly and if not, what is the problem.

With each class, there are three smoothie makers and one agent. Each smoothie maker runs in a loop calling `make` repeatedly (one with 0, one with 1, and one with 2). The agent runs in a loop calling `placeIngredients`.

You may ignore `InterruptedException`. Solutions without appropriate justifications will not receive marks.

(Part a). (6p) Analyse this semaphore-based solution:

```
3   class SmoothieSemaphore {
4       private final Semaphore[] ingredients = {new Semaphore(permits:0),
5           new Semaphore(permits:0), new Semaphore(permits:0)};
6
7       public void make(int ingredient) throws InterruptedException {
8           for (int i=1 ; i<3 ; ++i) {
9               int other = (ingredient + i) % 3;
10              ingredients[other].acquire();
11          }
12
13          System.out.println("Smoothie Maker " + ingredient + " is making a smoothie.");
14      }
15
16      public void placeIngredients(int ingredient1, int ingredient2) {
17              ingredients[ingredient1].release();
18              ingredients[ingredient2].release();
19      }
20  }
```

Answer:

This solution works as expected. Each semaphore counts the number of ingredients of each type.

(Part b). (6p) Analyse this monitor-based solution:

```
5   class SmoothieMonitor {
6       private final Lock lock = new ReentrantLock();
7       private final Condition[] conditions = {lock.newCondition(),
8           lock.newCondition(), lock.newCondition()};
9       private Integer[] ingredients = new Integer[3]; // [fruit, yogurt, juice]
10
11      public SmoothieMonitor() {
12          for (int i=0 ; i<3 ; ++i) {
13              ingredients[i]=0;
14          }
15      }
16
17      public void make(int ingredient) throws InterruptedException{
18          lock.lock();
19          try {
20              for (int i=1 ; i<3 ; ++i) {
21                  while (ingredients[(ingredient + i) % 3] == 0) {
22                      conditions[(ingredient + i) % 3].await();
23                  }
24              }
25
26              for (int i=1 ; i<3 ; ++i) {
27                  ingredients[(ingredient + i) % 3]--;
28              }
29
30              System.out.println("Smoothie Maker " + ingredient + " is making a smoothie.");
31          } finally {
32              lock.unlock();
33          }
34      }
35
36      public void placeIngredients(int ingredient1, int ingredient2) {
37          lock.lock();
38          try {
39              ingredients[ingredient1]++;
40              ingredients[ingredient2]++;
41              conditions[ingredient1].signal();
42              conditions[ingredient2].signal();
43          } finally {
44              lock.unlock();
45          }
46      }
47  }
```

Answer: This solution does not work as expected. The removal of the ingredients at lines 26-28 happens (potentially) a long time after the first ingredient has been ensured to be there in lines 21-23. Indeed, between checking that the number of ingredients is not 0 and taking the ingredient, there is potentially another call to await, which releases the lock and allows someone else to take the ingredient. This could lead the integer to fall below 0, corrupting the whole mechanism.

Q3 (12p). Here is the implementation of the producer-consumer solution in Erlang that was shown in class:

```
5   start(Bound) ->
6       spawn(fun () -> buffer([],0,Bound) end).
7
8   buffer(Content, Count, Bound) ->
9       receive
10          {get, Consumer, Ref} when Count > 0 ->
11              [ First | Rest ] = Content,
12              Consumer ! { item, Ref, First},
13              buffer(Rest,Count-1,Bound);
14          {put, Producer, Ref, Item } when Count < Bound ->
15              Producer ! { done, Ref },
16              buffer(Content++[Item], Count+1, Bound)
17      end.
18
19  get(Buffer) ->
20      Ref = make_ref(),
21      Buffer ! {get, self(), Ref},
22      receive { item, Ref, Item } -> Item
23      end.
24
25  put(Buffer,Item) ->
26      Ref = make_ref(),
27      Buffer ! {put, self(), Ref, Item},
28      receive {done,Ref} -> done
29      end.
```

We have criticized this implementation as it uses the unbounded mailbox of the process running the `buffer` function to implement a bounded buffer. In case that there is a real limit on the amount of memory allocated to one process and that items are large, this would be a real bottleneck. Here, you are going to change this implementation to ensure that indeed at most `Bound` items are kept at the buffer. You do this by spawning at most `Bound` (at any given time) bespoke "holder" processes containing at most one item each. The "holder" interacts with the producer and the consumer (get the item and deliver it).

(Part a). (3p)

Change the `put` method so that it sends a request to the buffer for the address of a "empty holder" (rather than sending an item). Then, it sends its item directly to this "holder".

(Part b). (3p)

Change the `get` method so that it sends a request to the buffer for the address of a "full holder" (rather than directly getting an item). Then,

when getting such an address, it communicates with it directly and gets an item from it.

(Part c). (2p)

Create a `holder` function that will be spawned by the buffer. It starts in "put" mode, where it await an item from the producer (matching your implementation of `put`) and then moves to "get" mode, where it awaits a contact from a consumer and then forward the item to them (matching your implementation of `get`).

(Part d). (4p)

Change the `buffer` method so that it spawns holders when they are needed upon interaction with producers, stores their addresses, and distributes them to consumers when they are ready to consume. Once the buffer sends the address of a holder to a consumer the item is considered out of the buffer.

Answer:

Here is an implementation of one possible solution:

```
8   buffer(Holders, Count, Bound) ->
9       receive
10          {get, Consumer, Ref} when Count > 0 ->
11              [ First | Rest ] = Holders,
12              {Holder, Tref} = First,
13              Consumer ! {holderg, Ref, Holder, Tref},
14              buffer(Rest,Count-1,Bound);
15          {put, Producer, Ref} when Count < Bound ->
16              Tref = make_ref(),
17              Holder = spawn(fun () -> holderp(self(), Producer, Tref) end),
18              Producer ! { holderp, Ref, Holder, Tref },
19              buffer(Holders++[{Holder,Tref}], Count+1, Bound)
20      end.
21
22  holderp(Buffer, Producer, Tref) ->
23      receive
24          { puth, Producer, Tref, Item } ->
25              Producer ! { done, Tref},
26              holderg(Buffer, Item,Tref)
27      end.
28
29  holderg(_Buffer, Item, Tref) ->
30      receive
31          { geth, Consumer, Tref } ->
32              Consumer ! { item, Tref, Item }
33      end.
```

```
35  get(Buffer) ->
36      Ref = make_ref(),
37      Buffer ! {get, self(), Ref},
38      receive
39          { holderg, Ref, Holder, Tref} ->
40              Holder ! { geth, self(), Tref },
41              receive
42                  { item, Tref, Item } ->
43                      Item
44              end
45      end.
46
47  put(Buffer,Item) ->
48      Ref = make_ref(),
49      Buffer ! {put, self(), Ref},
50      receive {holderp, Ref, Holder, Tref} ->
51          Holder ! { puth, self(), Tref, Item },
52          receive
53              { done, Tref } ->
54                  done
55          end
56      end.
```

Q4 (10p). You are supposed to implement a *queue* using a linked list as the underlying data structure. Your manager asks you to refactor an old code of a fine-grained locking version of a parallel linked set (code shown in Figures 6 and 7). Your original task is to implement a *bounded queue*, that is, you need to define a class `Queue<T>` as specified below.

Background: A *queue* is a FIFO (First In, First Out) data structure with the following operations:

`enqueue(Q,E)`: It adds element `E` to the queue `Q`. If the queue is *full*, then no element can be added (there is an *Overflow* condition). It gives as result the updated new queue with the new element added, or the very same queue in case of an Overflow condition.

`dequeue(Q)`: It removes an element from the queue `Q`. The elements are popped (dequeued) in the same order in which they are pushed (enqueued). If the queue is empty, no element is given (there is an *Underflow condition*). Otherwise, it gives as result the dequeued element and the new queue without the removed element.

`front(Q)`: It gets the front element of the queue `Q` without removing it. (That is, it gives the "oldest" element in the queue.)

`last(Q)`: It gets the last element of the queue `Q` without removing it. (That is, it gives the "newest" element in the queue.)

A queue is said to be *bounded* when there is a limit on the number of elements it might contain; we call the maximum number of elements the queue may contain its *bound* (or *limit*).

A queue is *full* when it has as many elements as its limit. Note that you can only enqueue a new element when the queue is not *full*).

For bounded queues, we have the following new operation:

`bound(Q)`: It gives the bound of the queue `Q` (the maximum number of elements the queue may contain).

In what follows you will get 10 assertions concerning the implementation of a class `Queue<T>` that was designed for parallel access as well as general questions for parallel data structures based on linked lists and about the possibility of reusing the code for sets (the code shown in Figures 6 and 7): refactoring `FineSet<T>` into a new class `Queue<T>`.

For each assertion, you need to state whether it is correct or not. You need to justify your answer in each case. NOTE: An answer without a justification will not be granted full points.

1 CAS (*compare-and-set*) is one of the techniques used to implement correct concurrent access for data structures based on linked lists. This technique requires the use of a *validation* step and atomic access to the *valid* and *next* attributes.

Answer: True: As explained in the lecture on linked list when discussing lock-free access: atomic access is required in the validation in order to avoid simultaneous access to the validation flag and the update of the next link.

2 It is possible to implement a class `Queue<T>` allowing for parallel access using CAS (compare-and-set) operation.

Answer: True: You can, as shown in the lectures for different data structures.

3 A race condition is possible when calling the `bound` method. It is thus mandatory to use a lock (or any other synchronisation mechanism) if accessed by more than one thread.

Answer: False: the `bound` just returns a value that is not supposed to be updated anywhere (and thus it doesn't require to be protected with any sync mechanism).

4 If you implement the class `Queue<T>` using a "lazy node removal" technique, then enqueuing (adding) an element is lock-free.

Answer: False: Adding elements on any parallel data structure using a "lazy node removal" technique requires the use of locks for adding an element (only the `has` is lock-free).

5 To implement the `enqueue` method it is possible to simply modify the `add` method from the `FineSet<T>` class by just renaming the function and its parameters and letting the rest of the code as it is. You may need to do other changes in other functions in order for this method to work correctly. If you answer True then explain how the refactoring works (what other functions are to be modified). If you answer False, explain why it is not possible.

Answer: True: You could reuse the code of the `add` almost as it is. You may need to change `find` so the addition is done in the right place and add a check when you call `enqueue` to be sure you don't add an element to a full queue.
ALTERNATIVE ANSWER: False: because you need to add a check for the queue is full, and you could remove the find method and always add an element at the end of the queue.

6 You cannot implement a parallel queue data structure without the use of a `key` (you will have a problem adding the elements in the right position otherwise).

Answer: False: You don't need to use the key as the elements don't need to be added in order according to the key. The keys are used for efficiency reasons.

7 Unlike a set, the removal of an element in a parallel queue (`dequeue` method) does not require the use of a synchronisation mechanism (e.g., a lock) since the element is always removed from the very same place each time (the front of the queue).

Answer: False: It is true that the `dequeue` *method only removes elements from one end of the data structure, but you still need to use a synchronisation mechanism while operating on the queue to avoid inconsistencies.*

8 In a correct implementation of the class `Queue<T>`, there is no need that the methods `front` and `last` are mutually exclusive as they will never end up accessing the same element, so it is safe to implement them without using locks (or any other synchronisation mechanism).

Answer: False: When the queue has exactly one element both methods access the same element, and some inconsistencies may happen if a another thread is adding or removing an element of the queue.

9 We have seen that you can implement a parallel data structure based on linked lists using so-called *optimistic locking*. If you implement the class `Queue<T>` using optimistic locking then you cannot guarantee starvation freedom.

Answer: True: In optimistic locking the method find is done without using a lock, using instead validation*. A thread t may fail validation forever if other threads keep removing and adding pred/curr between when t performs find and when it locks pred and curr.*

10 Another technique to implement a parallel data structure based on linked lists is to do it using a technique called *lazy node removal*. When using such a technique, you don't need to use locks as the *validation* is enough to guarantee that inconsistencies will not arise.

Answer: False: In the lazy node removal technique validation is not enough to guarantee consistency. The operations of adding and removing nodes still require locking.

```
1 package sets;
2
3 public class FineSet<T> extends SequentialSet<T>
4 {
5     public FineSet() {
6         super();
7     }
8
9     @Override
10    protected Position<T> find(Node<T> start, int key) {
11        Node<T> pred, curr;
12        pred = start;
13        pred.lock();
14        curr = start.next();
15        curr.lock();
16        while (curr.key() < key) {
17            pred.unlock();
18            pred = curr;
19            curr = curr.next();
20            curr.lock();
21        }
22        return new Position<T>(pred, curr);
23    }
24
25    @Override
26    public boolean add(T item) {
27        Node<T> node = newNode(item);
28        Node<T> pred = null, curr = null;
29        try {
30            Position<T> where = find(head, node.key());
31            pred = where.pred;
32            curr = where.curr;
33            return rawAdd(pred, curr, node);
34        } finally {
35            pred.unlock();
36            curr.unlock();
37        }
38    }
39
40 \\ code continues in Figure 12.
```

Figure 6: A "fine-grained locking" implementation of parallel linked sets.

```
1     @Override
2     public boolean remove(T item) {
3         int key = item.hashCode();
4         Node<T> pred = null, curr = null;
5         try {
6             Position<T> where = find(head, key);
7             pred = where.pred;
8             curr = where.curr;
9             return rawRemove(pred, curr, key);
10        } finally {
11            pred.unlock();
12            curr.unlock();
13        }
14    }
15
16    @Override
17    public boolean has(T item) {
18        int key = item.hashCode();
19        Node<T> pred = null, curr = null;
20        try {
21            Position<T> where = find(head, key);
22            pred = where.pred;
23            curr = where.curr;
24            return rawHas(curr, key);
25        } finally {
26            pred.unlock();
27            curr.unlock();
28        }
29    }
30
31    @Override
32    protected Node<T> newNode(int key) {
33        return new LockableNode<>(key);
34    }
35 }
```

Figure 7: A "fine-grained locking" implementation of parallel linked sets. [CONT.]